

DataTable, DataSet, DataRow, and List are data structures used for handling and manipulating data.

1.DataTable:

- DataTable is a part of the ADO.NET library and represents an in-memory, tabular structure of data.
- It consists of rows and columns, much like a database table.
- You can define the structure of the DataTable by adding columns, and then add rows with data.
- It's often used for working with relational data.

```
DataTable dataTable = new DataTable("MyTable");
```

// Adding columns

```
dataTable.Columns.Add("ID", typeof(int));  
dataTable.Columns.Add("Name", typeof(string));
```

// Adding rows

```
DataRow row = dataTable.NewRow();  
row["ID"] = 1;  
row["Name"] = "John";  
dataTable.Rows.Add(row);
```

2.DataSet:

- DataSet is also part of ADO.NET and is an in-memory representation of a collection of data, including DataTable objects.
- It can hold multiple tables, relationships, and constraints.

```
DataSet dataSet = new DataSet("MyDataSet");
```

// Adding DataTable to DataSet

```
dataSet.Tables.Add(dataTable);
```

3.DataRow:

- DataRow represents a row of data in a DataTable.
- You can access the values in a row using column names or indices.

```
DataRow firstRow = dataTable.Rows[0];  
int id = (int)firstRow["ID"];  
string name = (string)firstRow["Name"];
```

4.List:

- List is a generic collection in C# found in the System.Collections.Generic namespace.
- It is not specifically designed for tabular data but is more general-purpose for storing and manipulating sequences of elements.

```
List<string> names = new List<string>();  
names.Add("Alice");  
names.Add("Bob");
```

5.If you want a list of a custom type:

```
public class Person  
{  
    public int ID { get; set; }  
    public string Name { get; set; }  
}  
  
class Program  
{  
    static void Main()  
    {  
        List<Person> people = new List<Person>();  
  
        // Adding only one person  
        people.Add(new Person { ID = 1, Name = "Charlie" });  
  
        // Print the list for verification  
        foreach (var person in people)  
        {  
            Console.WriteLine($"ID: {person.ID}, Name: {person.Name}");  
        }  
    }  
}
```

OR

Store Multiple Records

```
public class Person
{
    public int ID { get; set; }
    public string Name { get; set; }
}

class Program
{
    static void Main()
    {
        List<Person> people = new List<Person>();

        people.Add(new Person { ID = 1, Name = "Charlie" });
        people.Add(new Person { ID = 2, Name = "Alice" }); // Adding a second person

        // Print the list for verification
        foreach (var person in people)
        {
            Console.WriteLine($"ID: {person.ID}, Name: {person.Name}");
        }
    }
}
```

If you want to add multiple records to the `people` list at once, you can use the `AddRange` method. Here's an example:

```
using System;
```

```
using System.Collections.Generic;
```

```
public class Person
```

```
{
```

```
    public int ID { get; set; }
```

```
    public string Name { get; set; }
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        List<Person> people = new List<Person>();
```

```
        // Adding multiple persons at once
```

```
        people.AddRange(new List<Person>
```

```
        {
```

```
            new Person { ID = 1, Name = "Charlie" },
```

```
            new Person { ID = 2, Name = "Alice" },
```

```
            new Person { ID = 3, Name = "Bob" }
```

```
            // Add more persons as needed
```

```
        });
```

```
        // Print the list for verification
```

```
        foreach (var person in people)
```

```
        {
```

```
            Console.WriteLine($"ID: {person.ID}, Name: {person.Name}");
```

```
        }
```

```
    }
```

```
}
```

Example: DataTable, DataSet, DataRow

```
using System;
using System.Data;

class Program
{
    static void Main()
    {
        // Create DataSet
        DataSet ds = new DataSet("CompanyDataSet");

        // Create Department Table
        DataTable deptTable = new DataTable("Department");
        deptTable.Columns.Add("DeptId", typeof(int));
        deptTable.Columns.Add("DeptName", typeof(string));
        deptTable.PrimaryKey = new DataColumn[] { deptTable.Columns["DeptId"] };

        // Create Employee Table
        DataTable empTable = new DataTable("Employee");
        empTable.Columns.Add("EmpId", typeof(int));
        empTable.Columns.Add("EmpName", typeof(string));
        empTable.Columns.Add("Salary", typeof(decimal));
        empTable.Columns.Add("DeptId", typeof(int));
        empTable.PrimaryKey = new DataColumn[] { empTable.Columns["EmpId"] };

        // Add tables to DataSet
        ds.Tables.Add(deptTable);
        ds.Tables.Add(empTable);

        // Insert Department Records
        deptTable.Rows.Add(1, "HR");
        deptTable.Rows.Add(2, "IT");
        deptTable.Rows.Add(3, "Finance");

        // Insert Employee Records
        empTable.Rows.Add(101, "Amit", 40000, 1);
        empTable.Rows.Add(102, "Neha", 50000, 2);
```

```

empTable.Rows.Add(103, "Rahul", 60000, 2);
empTable.Rows.Add(104, "Sneha", 45000, 3);
empTable.Rows.Add(105, "Vikas", 55000, 1);

// Display Employees
Console.WriteLine("Employee Details:");
foreach (DataRow row in empTable.Rows)
{
    Console.WriteLine($"{row["EmpId"]} {row["EmpName"]} {row["Salary"]}
{row["DeptId"]}");
}

// Update Salary
DataRow updateRow = empTable.Rows.Find(103);
updateRow["Salary"] = 65000;

// Delete Employee
DataRow deleteRow = empTable.Rows.Find(105);
empTable.Rows.Remove(deleteRow);

// Create Relation
DataRelation relation = new DataRelation(
    "DeptEmp",
    deptTable.Columns["DeptId"],
    empTable.Columns["DeptId"]
);
ds.Relations.Add(relation);

// Display Department-wise Employees
Console.WriteLine("\nDepartment-wise Employees:");
foreach (DataRow dept in deptTable.Rows)
{
    Console.WriteLine("Department: " + dept["DeptName"]);
    foreach (DataRow emp in dept.GetChildRows("DeptEmp"))
    {
        Console.WriteLine("  " + emp["EmpName"]);
    }
}
}
}

```

Summary:

These data structures serve different purposes. DataTable and DataSet are typically used in database-related scenarios, while List is more general-purpose for collections of objects. DataRow is specific to handling rows in a DataTable. Each has its own strengths and use cases depending on the requirements of your application.